Microservice Architecture - Development and Demonstration of a Rest API

Contents

Introduction and overview of the API 3
Concepts used in this project 5
Entity Relationships 5
Date Handling 8
DTO Usage 9
Pagination Strategy 11
Error Handling Approach 12
Database Design 13
Sample API Responses 14
Sample API Responses- Creating a Clinic 15
Sample API Responses- Postman response showing paginated appointment list 16
Sample API Responses- Error Response (404 Not Found) 18
Challenges Encountered and Solutions 19
Github Link 19

Introduction and overview of the API

This project develops a **RESTful API** for managing a travel vaccination clinic using **Spring Boot.** The system allows clinics to register their details and manage vaccination appointments for travellers preparing to visit regions such as Asia or Africa. The aim of the project is to demonstrate core backend development concepts using a realistic healthcare-related example.

The API is built around two main entities, **Clinic** and **Appointment**, where one clinic can have many appointments. Each appointment stores patient details, vaccine type, and appointment date and time. This domain was chosen because it clearly demonstrates entity relationships, validation, date handling, pagination, and structured error handling, which are key requirements of the assignment.

The application follows a layered architecture with **controller**, **service**, **repository**, and **DTO components**. Controllers manage HTTP requests, services contain business logic, repositories interact with the database through JPA, and DTOs control the data exposed to API users. The API supports **CRUD operations**, **input validation**, meaningful HTTP responses, and filtering appointments by date range.

The system runs using an **H2 in-memory database**, which allows the application to start quickly without manual database setup. The database tables are automatically generated from the entity classes and can be viewed through the H2 console.

Overall, the project shows how a **Spring Boot REST API** can be designed using good software engineering practices while remaining simple to understand and easy to run using an **H2 in-memory database**.

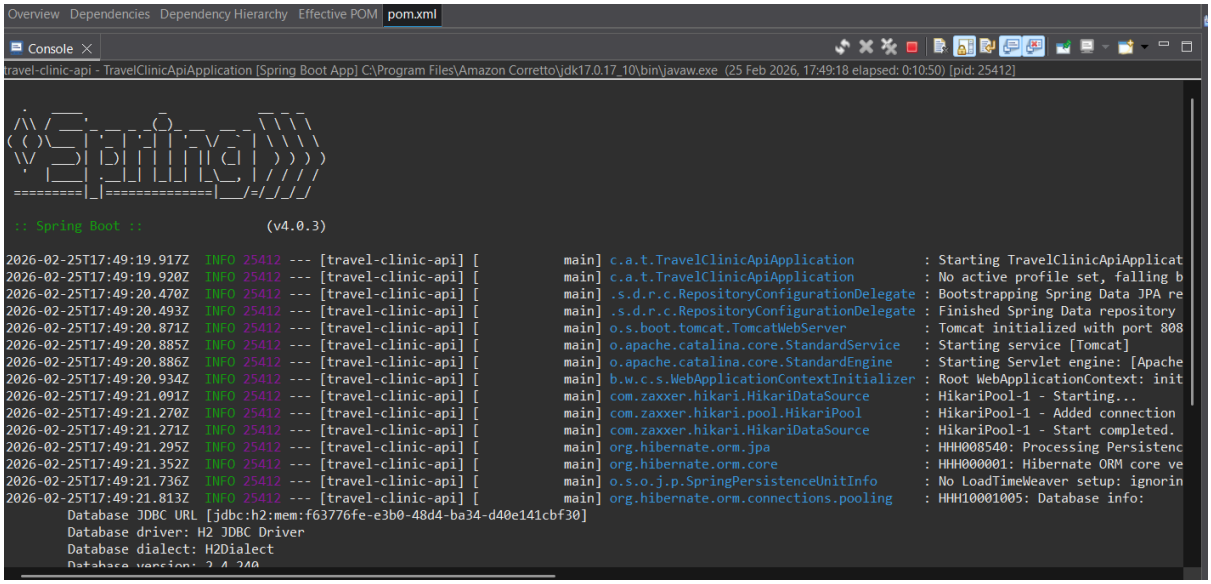

Figure 1: SpringBoot App is running on Eclipse

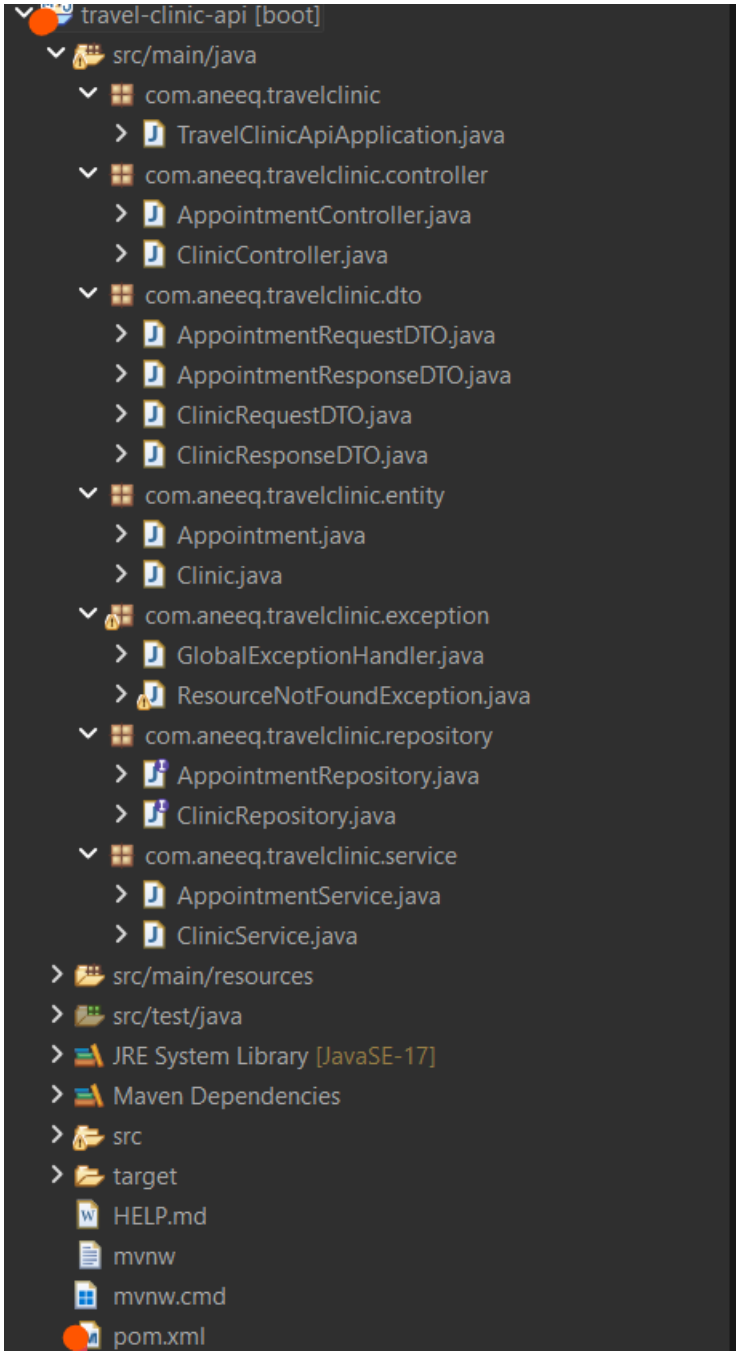

Figure 2: Project folder in structure in Eclipse

Concepts used in this project

Entity Relationships

The system is built around two main entities: **Clinic** and **Appointment**.

A clinic can manage multiple vaccination appointments, which creates a one-to-many relationship between the entities. Each appointment must belong to exactly one clinic.

In the database, this relationship is implemented using **JPA annotations**. The Clinic entity contains a list of appointments using the **@OneToMany annotation**, while the Appointment entity references a clinic using the **@ManyToOne annotation**. This ensures that appointments are correctly linked to their clinic and that the relationship is enforced at database level.

```
Clinic.java   Appointment...   AppointmentS...   AppointmentC...   travel-clini...   application...   SecurityConf...

6
7  @Entity
8  @Table(name = "clinics")
9  public class Clinic {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(nullable = false)
16     private String name;
17
18     @Column(nullable = false)
19     private String address;
20
21     @Column(nullable = false)
22     private String phone;
23
24     @Column(nullable = false)
25     private String email;
26
27     // One clinic -> many appointments
28     // orphanRemoval=true ensures removed child rows don't stay in DB
29     @OneToMany(mappedBy = "clinic", cascade = CascadeType.ALL, orphanRemoval = true)
30     private List<Appointment> appointments = new ArrayList<>();
31
32     public Clinic() {
33     }
34
35     public Clinic(String name, String address, String phone, String email) {
36         this.name = name;
37         this.address = address;
38         this.phone = phone;
39         this.email = email;
40     }
```

Figure 3: Clinic.Java

Figure 3 (Above) shows the Clinic entity class with the **@OneToMany** mapping to appointments.

This relationship also includes **cascading operations**. When a clinic is deleted, its related appointments are automatically removed using **cascade = CascadeType.ALL** and **orphanRemoval = true**. This prevents orphan records from remaining in the database and keeps data consistent.

```java
@Entity
@Table(name = "appointments")
public class Appointment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String patientName;

    @Column(nullable = false)
    private String patientEmail;

    @Column(nullable = false)
    private String vaccineType;

    @Column(nullable = false)
    private LocalDateTime appointmentDateTime;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "clinic_id", nullable = false)
    private Clinic clinic;

    public Appointment() {
    }

    public Appointment(String patientName,
                       String patientEmail,
                       String vaccineType,
                       LocalDateTime appointmentDateTime,
                       Clinic clinic) {
        this.patientName = patientName;
        this.patientEmail = patientEmail;
        this.vaccineType = vaccineType;
        this.appointmentDateTime = appointmentDateTime;
        this.clinic = clinic;
```

Figure 4: Appointment.java

In the H2 database, this relationship appears as a foreign key constraint between the appointments and clinics tables. The appointments table includes a clinic_id column that references the clinic's primary key.

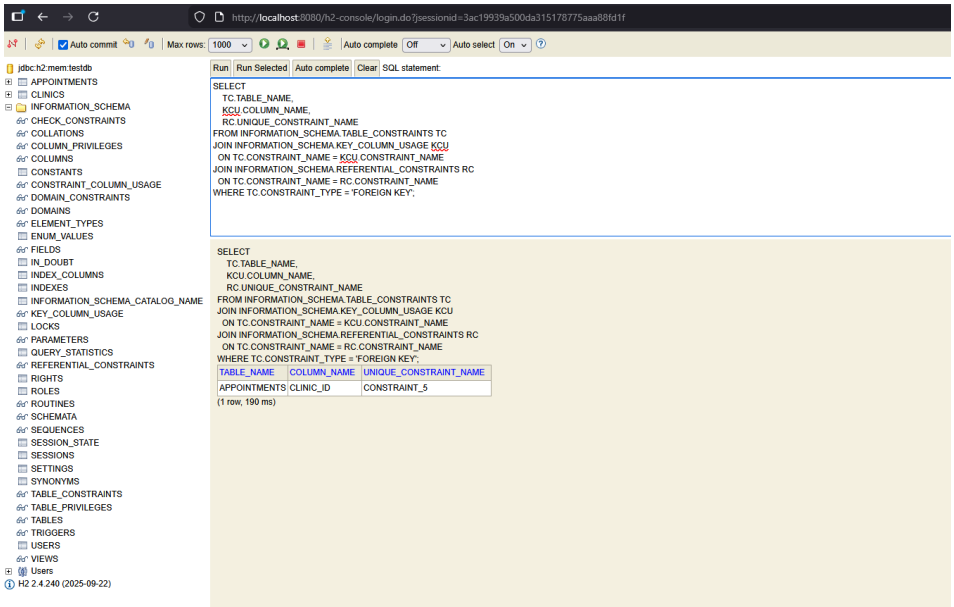

Figure 5: Shows the H2 Database tables with foreign key relationship

The API also allows navigating this relationship through endpoints such as:

- Retrieving all appointments for a specific clinic
- Creating appointments under a clinic
- Deleting a clinic and cascading deletion to its appointments

Date Handling

Appointments in the system include a date and time field called **appointmentDateTime**.

This is stored in the **Appointment** entity using the Java type **LocalDateTime**, which comes from the Java Time API (**java.time**). Using **LocalDateTime** avoids problems with old Date classes and provides clearer handling of time values.

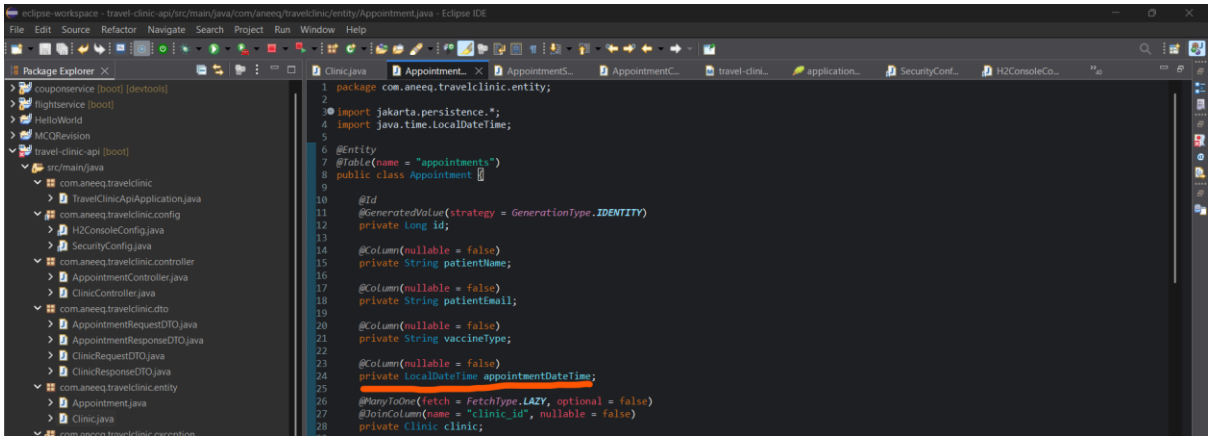

Figure 6: Appointment entity showing LocalDateTime field

When the API receives data from a client (for example through Postman as shown in Figure 6 below), the date is passed in ISO-8601 format such as **2026-03-10T14:30:00**. Spring Boot automatically converts this JSON value into a **LocalDateTime** object using Jackson. This means the controller does not need manual parsing.

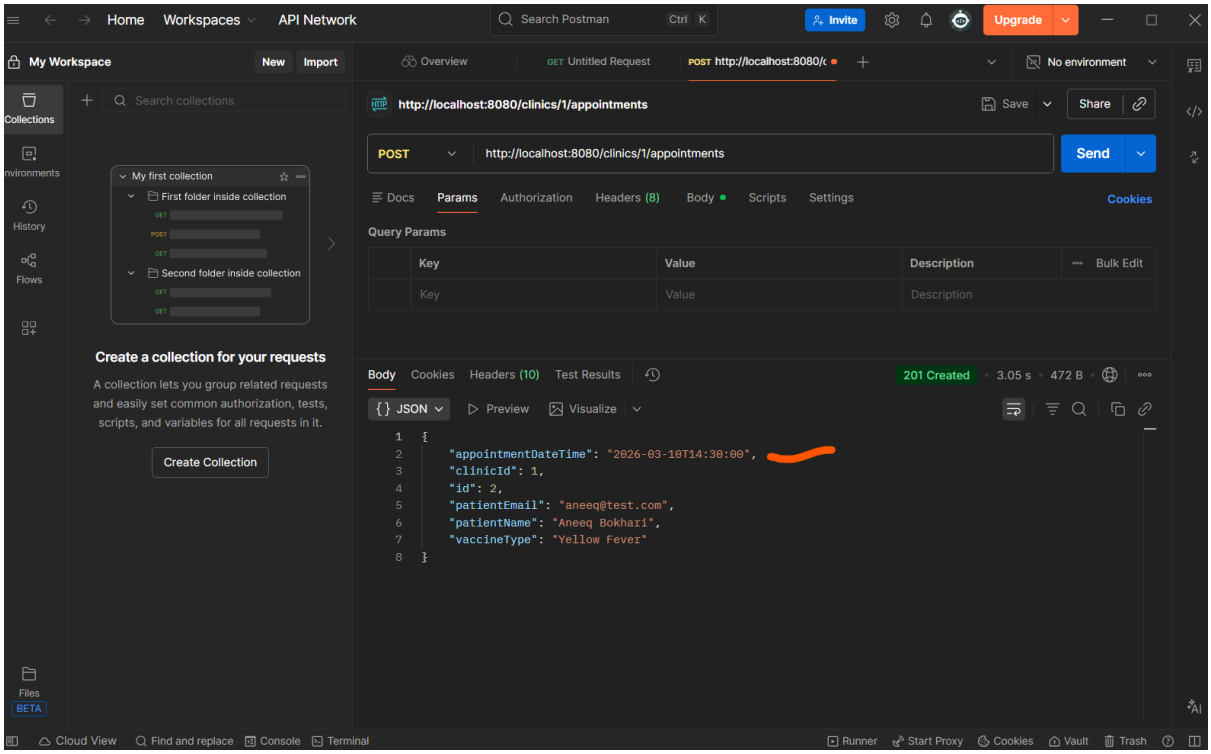

Figure 7: Appointment POST request with ISO-8601 date format

Inside the database, Hibernate maps **LocalDateTime** to a **TIMESTAMP** column in the **H2 database**.
You can see this in the H2 console where the **APPOINTMENTS** table shows the **APPOINTMENT_DATE_TIME** column.

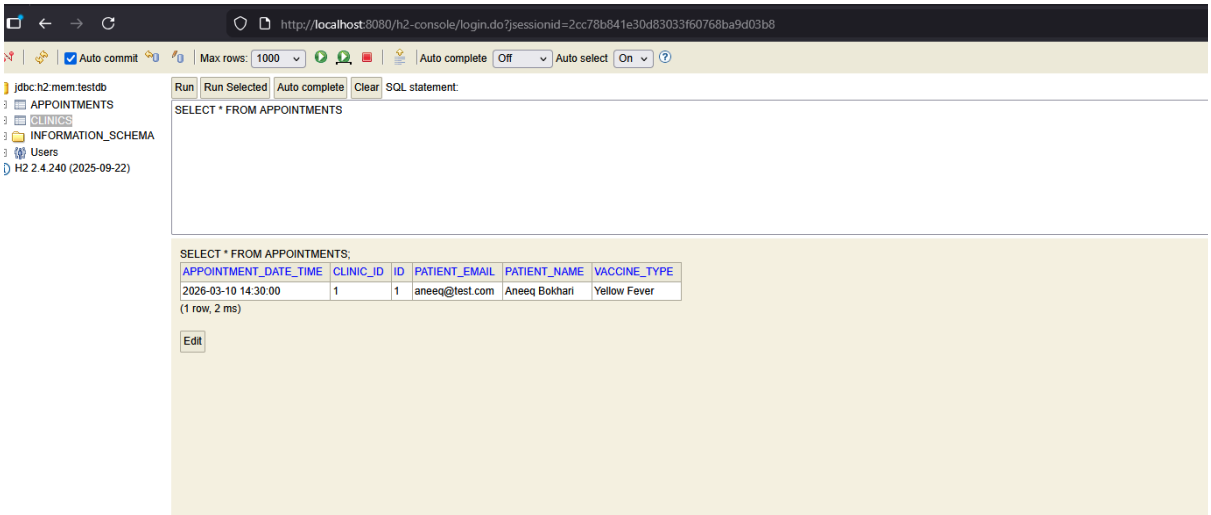

Figure 8: H2 database showing timestamp column (**Appointment_DATE_TIME** column)

The API also supports filtering appointments by date range. For example, users can request appointments between two dates using query parameters. This demonstrates practical handling of date comparisons in a healthcare context where scheduling accuracy is important.

Using **LocalDateTime** instead of strings improves validation, sorting, and database queries. It also avoids timezone mistakes and ensures the API behaves consistently.

DTO Usage

The project uses DTOs (**Data Transfer Objects**) to control what data is sent to and from the API.

Instead of exposing entity classes directly, separate DTO classes are used for requests and responses.

Examples in this project include:

- ClinicDTO
- AppointmentDTO
- AppointmentRequestDTO

These classes are located in the dto package.

DTOs help separate the database model from the API model.For example, the Appointment entity includes a full Clinic object, but the API request only needs the clinic ID. The DTO contains only the necessary fields such as **patient name, email**, **vaccine type**, and

appointment date. This prevents sending unnecessary data and avoids circular JSON problems between Clinic and Appointment.

Figures 9 and **10 (**Below**)** show the structure of the request and response DTO classes used in the project.

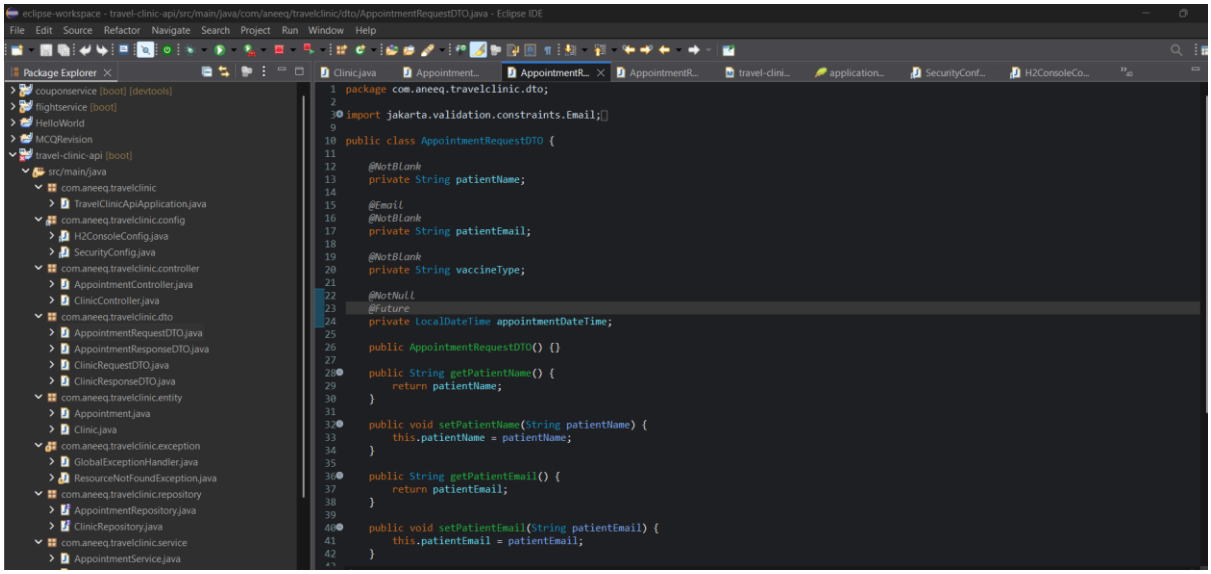

Figure 9: Appointment DTO structure (AppointmentRequestDTO.java)

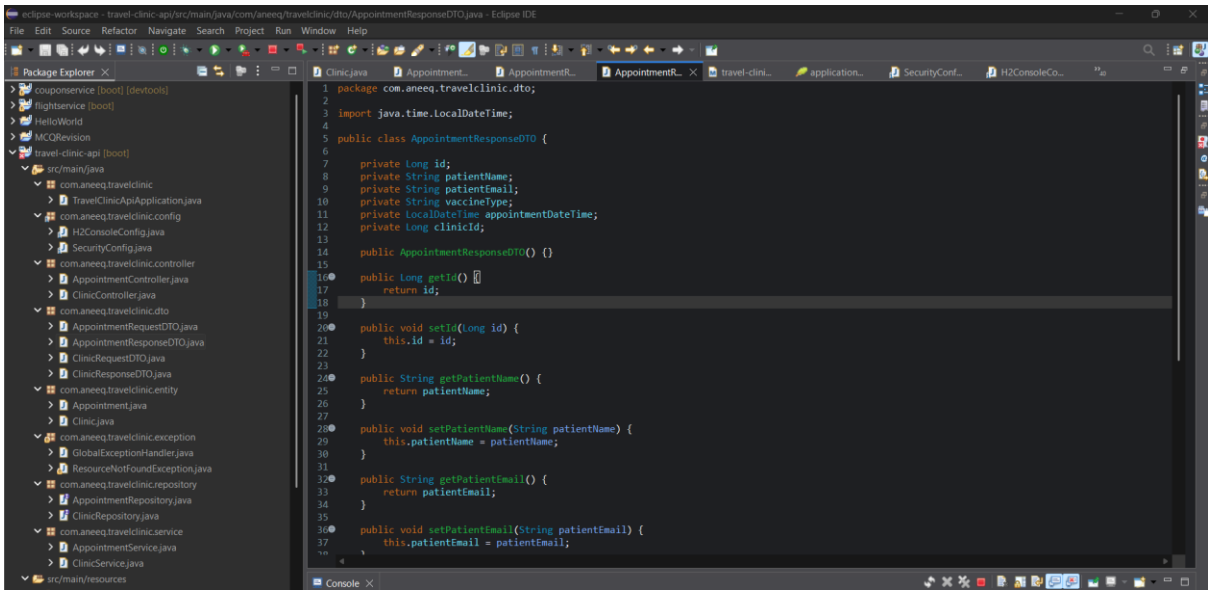

Figure 10: Appointment DTO structure (AppointmentRespondDTO.java)

DTOs also improve validation. Validation annotations such as **@NotBlank**, **@Email**, or **@Future** can be placed on DTO fields to check user input before it reaches the service layer. This keeps controllers clean and prevents invalid data entering the database.

The service layer converts DTOs into entity objects before saving them using the repository. This keeps the code organised and follows a layered architecture design.

Using DTOs makes the API easier to maintain, safer to use, and closer to real industry practice where backend systems must protect internal database structures.

Pagination Strategy

Some API endpoints can return many records, especially when listing appointments for a busy clinic. To keep responses fast and easier to manage, **pagination** is implemented using Spring Data JPA's Pageable interface.

Instead of returning the full list, the controller accepts query parameters such as page and size. These values are passed to the repository layer, which returns a **Page<AppointmentResponseDTO>** object. This object contains only a subset of records in the content list, along with useful metadata such as **totalElements**, **totalPages**, and **pageNumber**. This approach improves performance, reduces memory usage, and allows client applications to load data step-by-step.

For example, the endpoint ' **GET /clinics/1/appointments?page=0&size=5** ' returns only five appointments at a time for a specific clinic. This is important in real healthcare systems where a clinic may store thousands of appointment records.

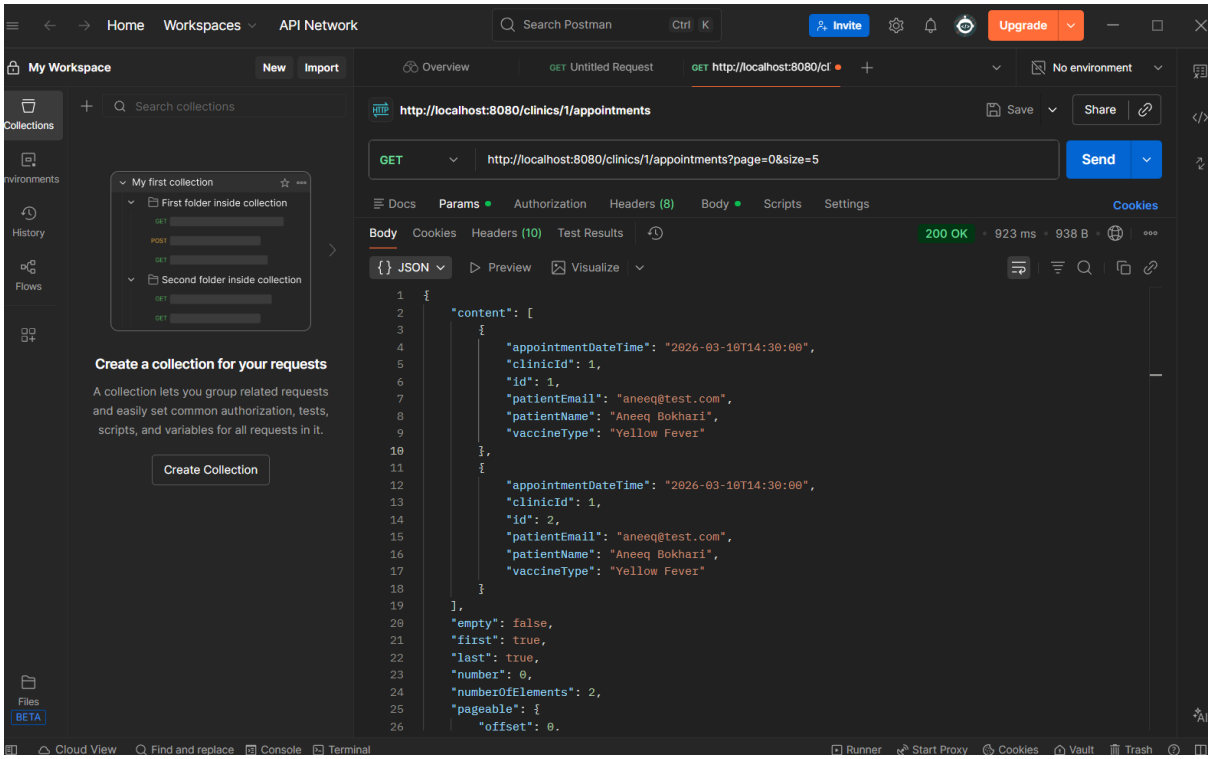

Figure 11: Paginated appointments response using page and size parameters.

Error Handling Approach

The API uses a global exception handling strategy to provide clear and consistent error responses. Instead of returning generic server errors, a **GlobalExceptionHandler** class is implemented using **@ControllerAdvice**. This class catches common exceptions such as **ResourceNotFoundException**, validation errors, and bad requests.

When an error occurs, the API returns a structured JSON response containing timestamp, status code, error message, and request path. This makes debugging easier and provides useful feedback to API users.

For example, if a clinic ID does not exist, the system throws **ResourceNotFoundException**, and the client receives a **404 Not Found** response instead of a generic error page.

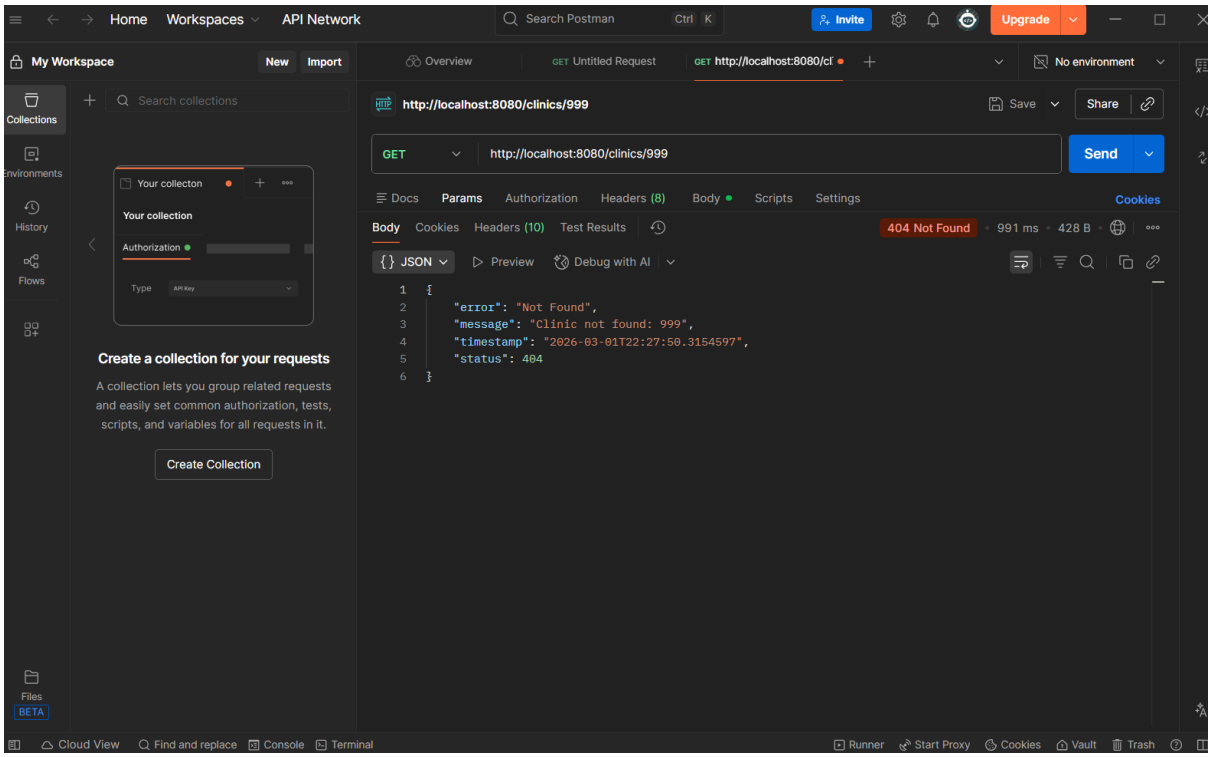

Figure 12: Postman response when requesting a non-existing clinic ID

Database Design

The database design for this project is based on two main tables: clinics and appointments. The ERD diagram shows the structure of these tables, including primary keys, foreign key relationships, and important fields.

The clinics table stores information about each clinic, such as name, address, phone number, and email. The primary key is id, which uniquely identifies each clinic.

The appointments table stores vaccination appointment details such as patient name, email, vaccine type, and appointment date/time. The primary key is id, and the table contains a foreign key clinic_id that links each appointment to a specific clinic. This creates a one-to-many relationship, where one clinic can have multiple appointments.

This design matches the entity classes in the project (**Clinic.java** and **Appointment.java**) and ensures data consistency through foreign key constraints.

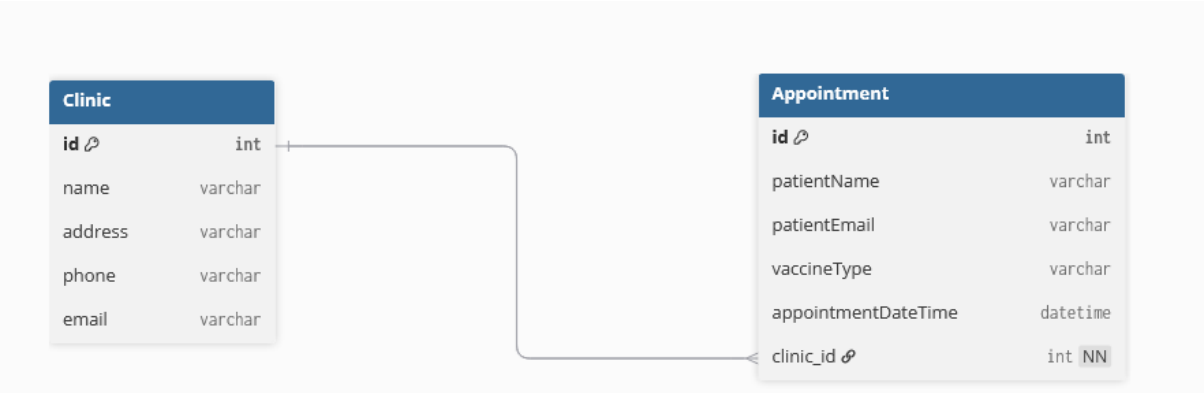

Figure 13: ERD diagram showing clinics and appointments tables with primary and foreign keys.

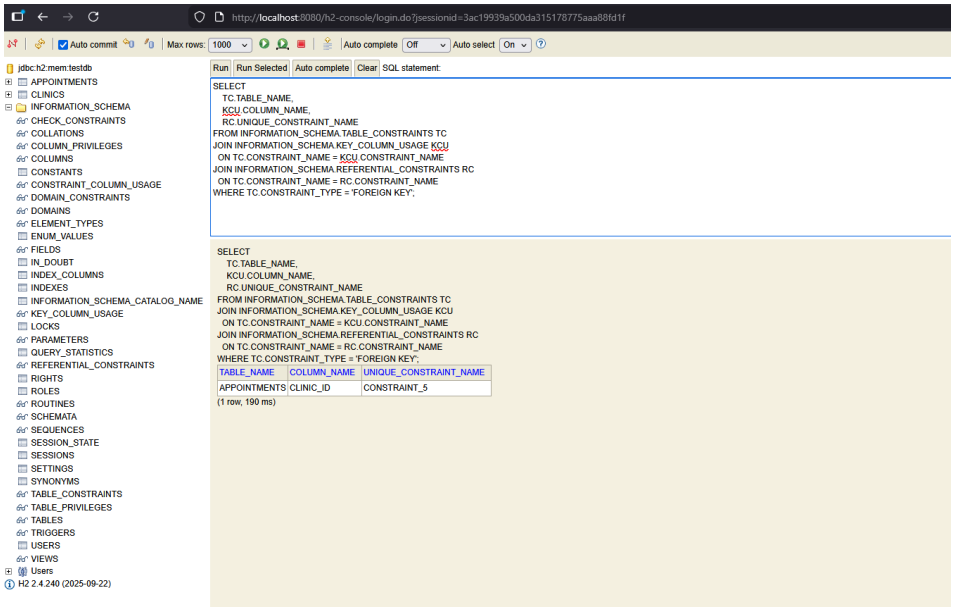

Figure 14: H2 database showing **clinic**_id foreign key constraint

Sample API Responses

The API returns JSON responses that follow standard REST conventions. These responses include entity data as well as metadata such as HTTP status codes and pagination details.

For example, when creating a clinic using **POST /clinics**, the API returns a JSON object with the clinic details, including the generated ID.

When listing appointments using **GET /clinics/1/appointments?page=0&size=5**, the API returns a paginated response. The response contains a content list with appointment records and metadata fields such as **totalElements**, **totalPages**, and **pageNumber**. This makes it easier for frontend applications to display data in pages.

The appointment object includes fields such as:

- appointmentDateTime
- clinicId
- patientName
- patientEmail
- vaccineType

Sample API Responses- Creating a Clinic

A new clinic can be registered by sending a POST request to the /clinics endpoint with clinic details in JSON format. The API validates the input fields and returns a response with HTTP status 201 Created together with the saved clinic information, including the generated clinic ID.

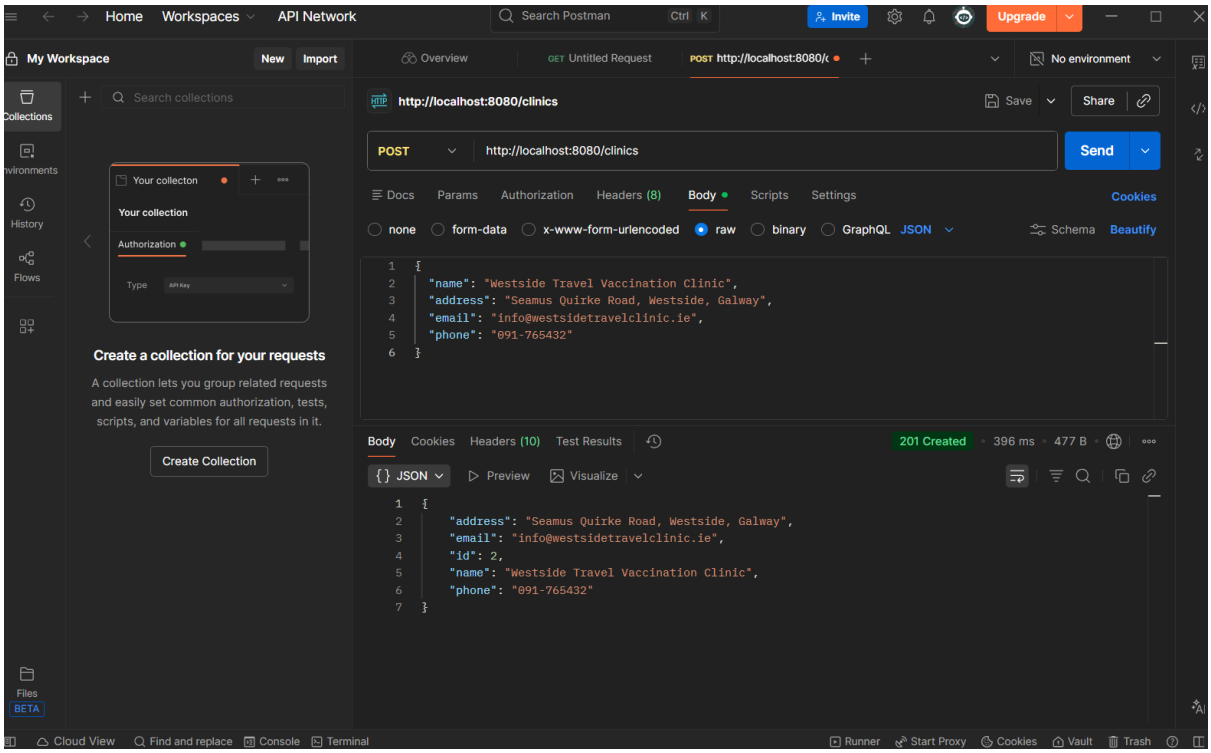

Figure 15: Postman POST request and response showing successful clinic creation

This response (**Figure 15** above) confirms that the clinic was successfully stored in the **H2 database** and assigned a unique identifier. The clinic ID is later used when creating appointments linked to that clinic.

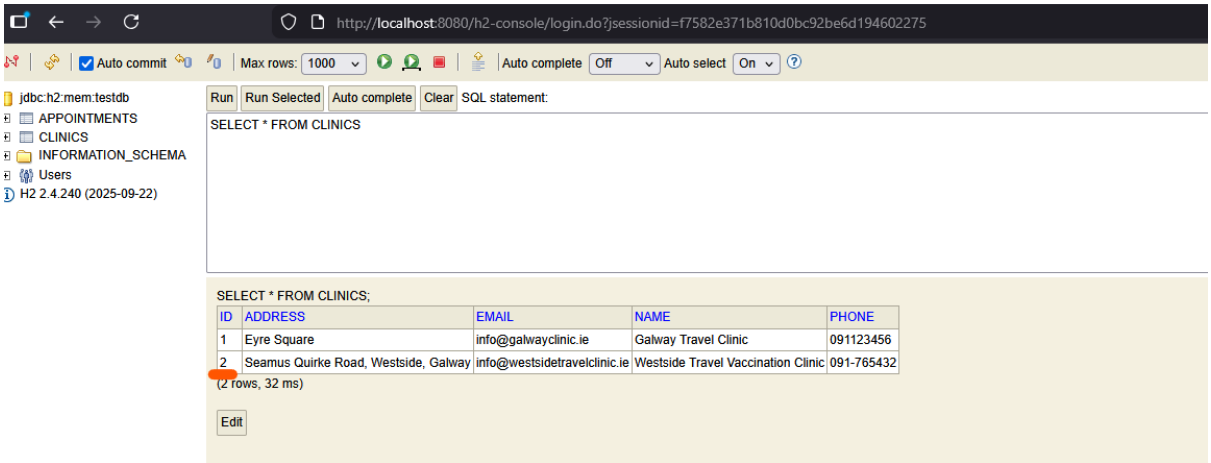

Figure 16: H2 Database confirmed Clinic (Westside Travel Vaccination Clinic) was created and assigned a unique identifier (id = 2)

Sample API Responses- Postman response showing paginated appointment list

When a clinic has many appointments, the API returns results in pages instead of sending the full list. **Pagination** is implemented using Spring Data JPA's Pageable interface. The client can request a specific page number and page size using query parameters.

For example, the request below retrieves the first five appointments for clinic ID 1.

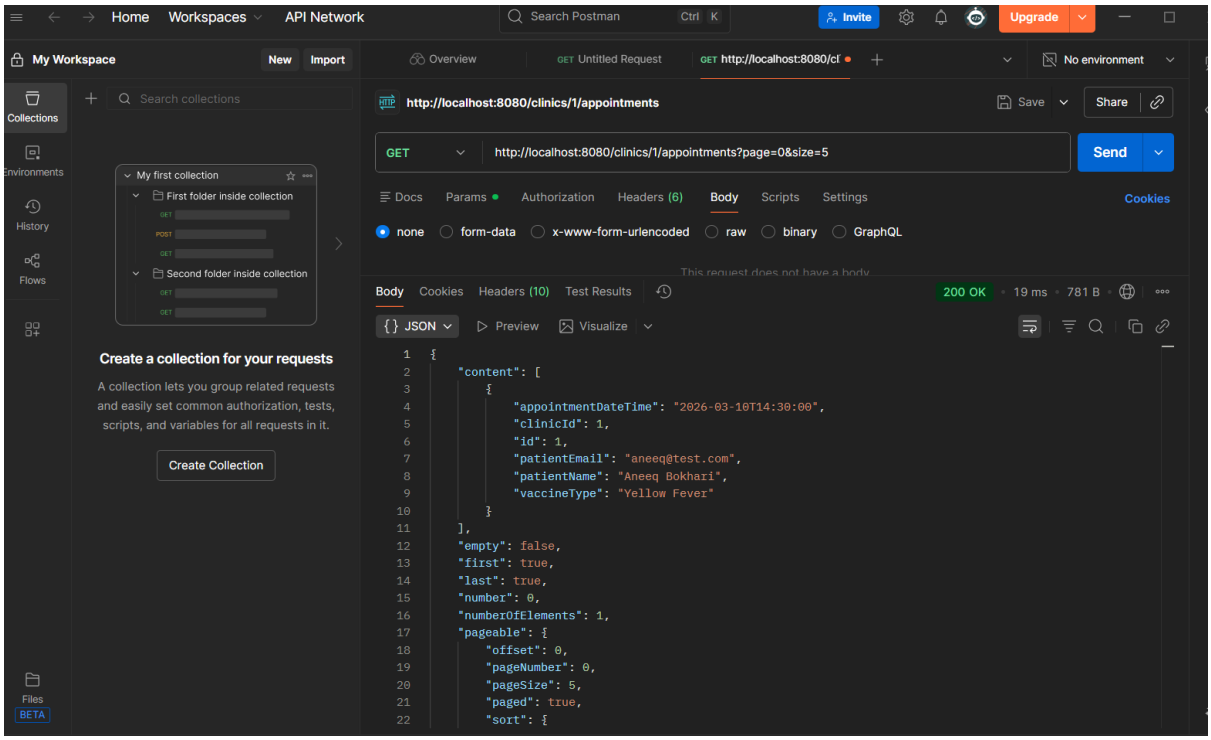

Figure 17: Postman response showing paginated appointment list

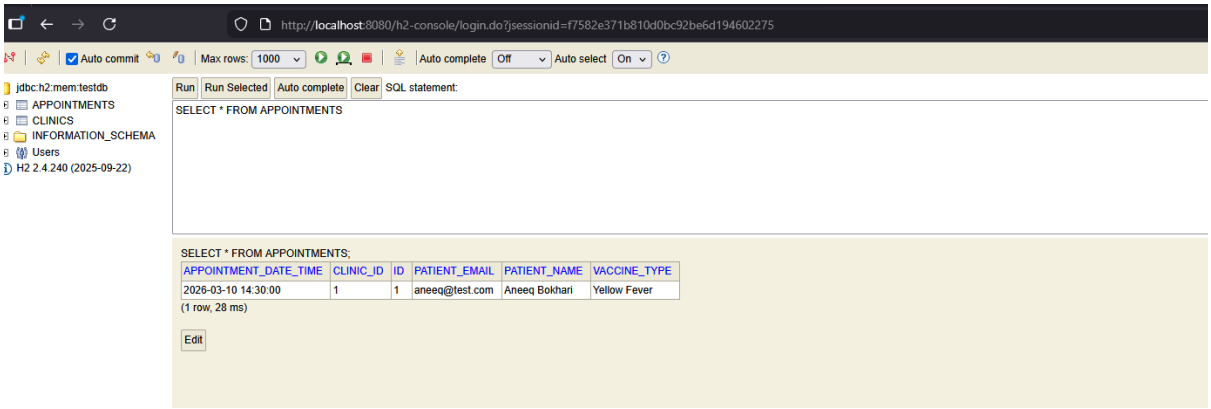

Figure 18: H2 Database showing the list of appointment in the '**APPOINTMENTS'** table

This response includes only a subset of records inside the content field. Additional metadata such as **totalElements**, **totalPages**, and **pageSize (**As shown in **Figure 19** below**)** allows the client application to display navigation controls and load further pages if needed.

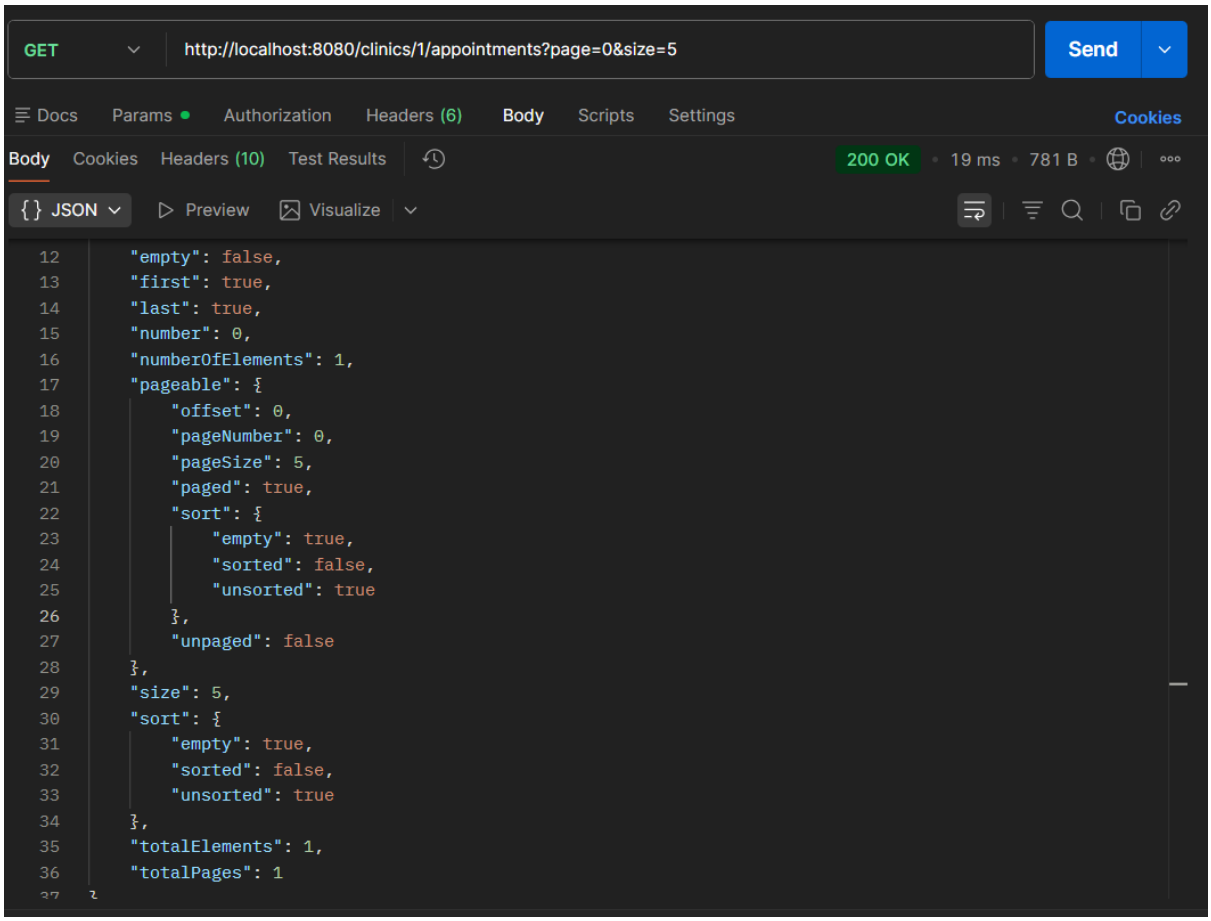

Figure 19: Postman response showing the rest of metadata in the paginated appointment list

Sample API Responses- Error Response (404 Not Found)

The API includes structured error handling so that clients receive clear and meaningful HTTP responses when something goes wrong. Spring Boot automatically returns standard status codes such as **400 Bad Request**, **404 Not Found**, and **405 Method Not Allowed**. In addition, a custom **ResourceNotFoundException** and a global exception handler are used to return consistent JSON error messages.

For example, if a client sends a request using the wrong HTTP method, the API returns a 405 Method Not Allowed response. In this project, the endpoint **/clinics/{clinicId}/appointments/{appointmentId}** supports **PUT** and **DELETE** but not **GET**. Sending a GET request to this URL results in a 405 error, which tells the client that the endpoint exists but the method is not supported.

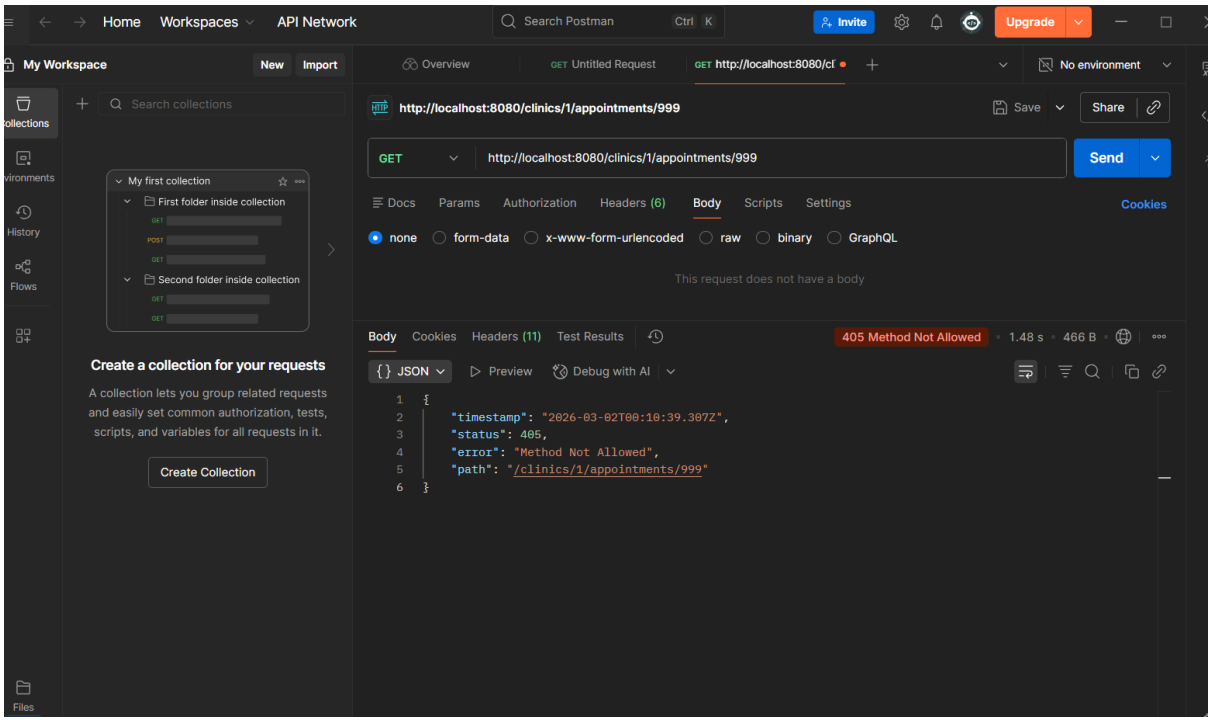

Figure 20: Postman response showing 405 ‘ **Method Not Allowed** ‘ error

This approach helps API users quickly understand what went wrong and how to fix their request.

Challenges Encountered and Solutions

During development, several issues were encountered while building and testing the API.

One challenge was configuring the H2 in-memory database. At first, the H2 console did not open due to security settings and dependency issues. This was fixed by adding the correct H2 dependency, configuring **application.properties**, and allowing **/h2-console/**** access in **SecurityConfig**.

Another issue was understanding how the in-memory database works. The database is created automatically at runtime and reset when the application restarts. This required checking data quickly using the H2 console before restarting Spring Boot.

Pagination also caused confusion initially, because the endpoint path included the clinic ID. The correct request format is /**clinics/{clinicId}/appointments?page=0&size=5.**

There were also some problems with **HTTP 403** and **404 errors** while testing in **Postman**. These were solved by updating the controller mappings and fixing Spring Security configuration.

Overall, these challenges helped improve understanding of Spring Boot configuration, REST endpoint testing, and database relationships.

Github Link

The full source code for the Travel Vaccination Clinic REST API project is available on GitHub at the following link:

https://github.com/aneeq123/travel-clinic-api-springboot

The repository is public and accessible to anyone, and the complete project files, documentation, and screenshots will be uploaded there.